ITA0605- MACHINE LEARNING (SLOT-D)

CO1- ASSESSMENT TOOL 1- APPLICATION BASED QUESTIONS

Dhivya R (192312619)

Date: 23/07/2026

SET 1

1. A factory has three boxes containing colored balls. Box A contains 6 red, 4 blue, and 2 green balls; Box B contains 5 red, 7 blue, and 3 green balls; and Box C contains 8 red, 2 blue, and 5 green balls. One box is selected at random, but Box B is twice as likely to be chosen as each of the other boxes. A ball drawn from the chosen box is found to be blue. What is the probability that the selected box was Box C?

```python
# Question 1: Probability that selected box was Box C

P_A = 1/4
P_B = 2/4
P_C = 1/4

P_Blue_A = 4 / (6 + 4 + 2)
P_Blue_B = 7 / (5 + 7 + 3)
P_Blue_C = 2 / (8 + 2 + 5)

P_Blue = (P_Blue_A * P_A) + (P_Blue_B * P_B) + (P_Blue_C * P_C)

P_C_given_Blue = (P_Blue_C * P_C) / P_Blue

print("Probability that selected box was Box C =", round(P_C_given_Blue, 4))
```

input

```
Probability that selected box was Box C = 0.0952


...Program finished with exit code 0
Press ENTER to exit console.
```

2. A parking facility has three sections containing cars of different colors. Section A has 5 red, 3 blue, and 2 black cars; Section B has 4 red, 6 blue, and 5 black cars; and Section C has 7 red, 2 blue, and 6 black cars. A section is chosen at random, but Section B is twice as likely to be selected as each of the other sections due to higher usage. A randomly selected car from the chosen section turns out to be blue. What is the probability that the car came from Section C?

```python
# Question 2: Probability that car came from Section C

P_A = 1/4
P_B = 2/4
P_C = 1/4

P_Blue_A = 3 / (5 + 3 + 2)
P_Blue_B = 6 / (4 + 6 + 5)
P_Blue_C = 2 / (7 + 2 + 6)

P_Blue = (P_Blue_A * P_A) + (P_Blue_B * P_B) + (P_Blue_C * P_C)

P_C_given_Blue = (P_Blue_C * P_C) / P_Blue

print("Probability that car came from Section C =", round(P_C_given_Blue, 4))
```

input

```
Probability that car came from Section C = 0.1081


...Program finished with exit code 0
Press ENTER to exit console.
```

3. In a real-world medical image classification task for detecting breast cancer, a deep-learning model integrated with the K-Nearest Neighbors algorithm is applied for binary classification (malignant vs. benign). The dataset consists of 1800 samples per class, with 20% of the data reserved for testing. During evaluation, the model reports 210 True Positives (TP), 190 True Negatives (TN), and 30 False Positives (FP). Based on the given information, determine the missing False Negatives (FN) value, and then calculate the Accuracy, Precision, Sensitivity (Recall), and Specificity of the classifier, explaining its effectiveness in medical diagnosis.

```python
# Question 3: Accuracy, Precision, Sensitivity, Specificity

samples_per_class = 1800
total_samples = samples_per_class * 2

test_samples = int(0.20 * total_samples)

TP = 210
TN = 190
FP = 30

FN = test_samples - (TP + TN + FP)

accuracy = (TP + TN) / (TP + TN + FP + FN)
precision = TP / (TP + FP)
sensitivity = TP / (TP + FN)
specificity = TN / (TN + FP)

print("Total test samples =", test_samples)
print("False Negatives (FN) =", FN)
print("Accuracy =", round(accuracy * 100, 2), "%")
print("Precision =", round(precision * 100, 2), "%")
print("Sensitivity / Recall =", round(sensitivity * 100, 2), "%")
print("Specificity =", round(specificity * 100, 2), "%")
```

```
Precision = 87.5 %
Sensitivity / Recall = 42.0 %
Specificity = 86.36 %


...Program finished with exit code 0
Press ENTER to exit console.
```

4. Examine the performance of a machine-learning model using its training and validation loss values over multiple epochs:

a. Epoch 1: Training Loss = 2.8, Validation Loss = 2.4

a. Epoch 2: Training Loss = 2.5, Validation Loss = 2.2

(c) Epoch 3: Training Loss = 2.2, Validation Loss = 2.0

(d) Epoch 4: Training Loss = 2.0, Validation Loss = 2.1

(e) Epoch 5: Training Loss = 1.8, Validation Loss = 2.3

(f) Epoch 6: Training Loss = 1.6, Validation Loss = 2.6

(g) Epoch 7: Training Loss = 1.5, Validation Loss = 2.9

(h) Epoch 8: Training Loss = 1.3, Validation Loss = 3.2

i) Identify the epoch at which the model begins to overfit.

ii) Suggest two effective techniques to reduce overfitting in this model.

iii) Discuss how overfitting affects the model's generalization ability on unseen data.

```python
# Question 4: Overfitting using loss values

training_loss = [2.8, 2.5, 2.2, 2.0, 1.8, 1.6, 1.5, 1.3]
validation_loss = [2.4, 2.2, 2.0, 2.1, 2.3, 2.6, 2.9, 3.2]

overfit_epoch = None

for i in range(1, len(validation_loss)):
    if training_loss[i] < training_loss[i - 1] and validation_loss[i] > validation_loss[i - 1]:
        overfit_epoch = i + 1
        break

print("Model begins to overfit at Epoch", overfit_epoch)

print("Two techniques to reduce overfitting:")
print("1. Early stopping")
print("2. Dropout or regularization")

print("Overfitting affects generalization because the model learns training data too much")
print("and performs poorly on new unseen data.")
```

input

```
2. Dropout or regularization
Overfitting affects generalization because the model learns training data too much
and performs poorly on new unseen data.
```

5. Examine the performance of a machine-learning model using training and validation accuracy over multiple epochs:

(a) Epoch 1: Training Accuracy = 60%, Validation Accuracy = 58%

(b) Epoch 2: Training Accuracy = 65%, Validation Accuracy = 63%

(c) Epoch 3: Training Accuracy = 70%, Validation Accuracy = 68%

(d) Epoch 4: Training Accuracy = 75%, Validation Accuracy = 72%

(e) Epoch 5: Training Accuracy = 80%, Validation Accuracy = 73%

(f) Epoch 6: Training Accuracy = 85%, Validation Accuracy = 72%

(g) Epoch 7: Training Accuracy = 88%, Validation Accuracy = 70%

(h)Epoch 8: Training Accuracy = 92%,Validation Accuracy = 68%

i) Identify the epoch at which the model starts overfitting.

ii) Explain why validation accuracy decreases despite training accuracy increasing.

iii) Suggest two methods to improve validation performance.

```python
# Question 5: Overfitting using accuracy values

training_accuracy = [60, 65, 70, 75, 80, 85, 88, 92]
validation_accuracy = [58, 63, 68, 72, 73, 72, 70, 68]

overfit_epoch = None

for i in range(1, len(validation_accuracy)):
    if training_accuracy[i] > training_accuracy[i - 1] and validation_accuracy[i] < validation_accuracy[i - 1]
        overfit_epoch = i + 1
        break

print("Model starts overfitting at Epoch", overfit_epoch)

print("Validation accuracy decreases because the model memorizes the training data")
print("instead of learning general patterns.")

print("Two methods to improve validation performance:")
print("1. Early stopping")
print("2. Data augmentation or regularization")
```

input

```
Two methods to improve validation performance:
1. Early stopping
2. Data augmentation or regularization
```